

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 2

Student: Deep S. (dns33)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 802.00/802.00 (100.00%)

Received Grade: 0.00/802.00 (0.00%)

Started: 4/19/2026 2:28:43 PM

Updated: 4/27/2026 3:39:58 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/grading/dns33>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-2/view/dns33>

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0
- 1. Refer to Milestone2 of this doc:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone2 branch
 - 1. `git checkout Milestone2` (ensure proper starting branch)
 - 2. `git pull origin Milestone2` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone2`
 - 4. On Github merge the pull request from Milestone2 to qa
 - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
 - 1. `git checkout qa`
 - 2. `git pull origin qa`

Section #1: (2 pts.) Data

Progress: 100%

≡ Task #1 (0 50 pts) - API Data Table

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/fields beyond `id`, `created`, `modified`, and respective `is_api` or `api_id` columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an `api_id` or `is_api` column)

The screenshot shows a database table named `tbl_videos` with the following structure:

Field	Type	Length	Null	Default	Extra
<code>id</code>	<code>INT(11)</code>		<code>NO</code>		<code>PRIMARY KEY</code>
<code>channel_id</code>	<code>VARCHAR(48)</code>		<code>NO</code>		
<code>title</code>	<code>VARCHAR(255)</code>		<code>NO</code>		
<code>length</code>	<code>INT(11)</code>		<code>NO</code>		
<code>published</code>	<code>TEXT</code>		<code>NO</code>		
<code>views</code>	<code>INT(11)</code>		<code>NO</code>		
<code>thumbnail</code>	<code>VARCHAR(255)</code>		<code>NO</code>		
<code>created</code>	<code>TIMESTAMP</code>		<code>NO</code>	<code>DEFAULT CURRENT_TIMESTAMP</code>	
<code>modified</code>	<code>TIMESTAMP</code>		<code>NO</code>	<code>DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP</code>	
<code>is_api</code>	<code>TINYINT(1)</code>		<code>NO</code>	<code>DEFAULT 1</code>	

Table - 1 Video

The screenshot shows the SQL code for creating the `tbl_videos` table with comments:

```

CREATE TABLE `tbl_videos` (
  `id` INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `channel_id` VARCHAR(48) NOT NULL,
  `title` VARCHAR(255) NOT NULL,
  `length` INT(11) NOT NULL,
  `published` TEXT NOT NULL,
  `views` INT(11) NOT NULL,
  `thumbnail` VARCHAR(255) NOT NULL,
  `created` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `modified` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `is_api` TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;
  
```

Comments on the right side of the code provide context for the fields:

- `channel_id`: e.g., UCpVcdtGKALCOWRjwag
- `title`: channel title
- `length`: video length in seconds
- `published`: video description
- `views`: video view count
- `thumbnail`: video thumbnail URL
- `created`: video creation timestamp
- `modified`: video modification timestamp
- `is_api`: video is API or not

sql-1 - Video

The screenshot shows the SQL code for creating the `tbl_channels` table:

```

CREATE TABLE `tbl_channels` (
  `id` INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `channel_id` VARCHAR(48) NOT NULL,
  `channel_title` VARCHAR(255) NOT NULL,
  `channel_desc` TEXT NOT NULL,
  `channel_thumbnail` VARCHAR(255) NOT NULL,
  `created` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `modified` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `is_api` TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;
  
```

sql-2 - Channel

The screenshot shows the SQL code for creating the `tbl_channels` table:

```

CREATE TABLE `tbl_channels` (
  `id` INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `channel_id` VARCHAR(48) NOT NULL,
  `channel_title` VARCHAR(255) NOT NULL,
  `channel_desc` TEXT NOT NULL,
  `channel_thumbnail` VARCHAR(255) NOT NULL,
  `created` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `modified` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `is_api` TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;
  
```

	id	channel_id	title	vanity_url	verified	subscribers_text	avatar_url	created	modified	is_active
	int	varchar(40)	varchar(120)	varchar(255)	boolean	varchar(40)	varchar(255)	timestamp	timestamp	tinyint(1)
1	1	UHLQJ101	My Video Channel	https://youtube.com/MyVideoChannel	0	100K subscribers	https://example.com/logo.jpg	2026-04-13 15:41:15	2026-04-13 15:41:15	0
2	2	UHLQJ102	My Sample Channel	https://youtube.com/MySampleChannel	1	100K subscribers	https://example.com/logo.jpg	2026-04-12 15:41:15	2026-04-12 15:41:15	1

Table -2 Channel

 Saved: 4/26/2026 11:00:00 AM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in `.sql`)

URL #1

https://github.com/shahdp2/IT202-SP2026/blob/Milestone2-Section1-Data/public_html/project/sql/006_create_table_yt_channels.sql



URL

https://github.com/shahdp2/IT202-SP2026/blob/Milestone2-Section1-Data/public_html/project/sql/006_create_table_yt_channels.sql



URL #2

https://github.com/shahdp2/IT202-SP2026/blob/Milestone2-Section1-Data/public_html/project/sql/007_create_table_yt_videos.sql



URL

https://github.com/shahdp2/IT202-SP2026/blob/Milestone2-Section1-Data/public_html/project/sql/007_create_table_yt_videos.sql



 Saved: 4/26/2026 11:00:00 AM

Part 3:

Progress: 100%

Details:

- Briefly note what each field/property represents

Your Response:

ALL YOUTUBE CRUD OPERATIONS ARE ADMIN ONLY IT202_M2_YT_Channels fields

- id: Internal auto-increment primary key for each channel record.
- channel_id: The YouTube channel's unique ID (from YouTube/RapidAPI). Used to prevent duplicates.
- title: The channel's display name/title.
- vanity_url: The channel's vanity URL (ex: youtube.com/@name) if available.
- verified: 0/1 flag showing whether the channel is verified.
- subscribers_text: Subscriber count as text from the API (ex: "3.54K subscribers").
- avatar_url: URL of the channel's avatar image (stored as a thumbnail URL).
- created: Timestamp when the row was created in the database.
- modified: Timestamp when the row was last updated.

- is_api: 1 if the record came from the API, 0 if it was manually created.

IT202_M2_YT_Videos fields

- id: Internal auto-increment primary key for each video record.
- video_id: The YouTube video's unique ID (from YouTube/RapidAPI). Used to prevent duplicates.
- channel_id: The channel ID this video belongs to (links videos back to a channel).
- title: The video title.
- channel_name: The channel name shown on the video result (useful for display).
- length_text: Video duration as text (ex: "4:37").
- published_text: Published time as text (ex: "1 year ago").
- views_text: View count as text (ex: "6,965 views").
- thumbnail_url: URL of the video thumbnail image.
- created: Timestamp when the row was created in the database.
- modified: Timestamp when the row was last updated.
- is_api: 1 if the record came from the API, 0 if it was manually created.



Saved: 4/26/2026 11:00:00 AM

≡ Task #2 (0.50 pts.) - Data Transformation

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

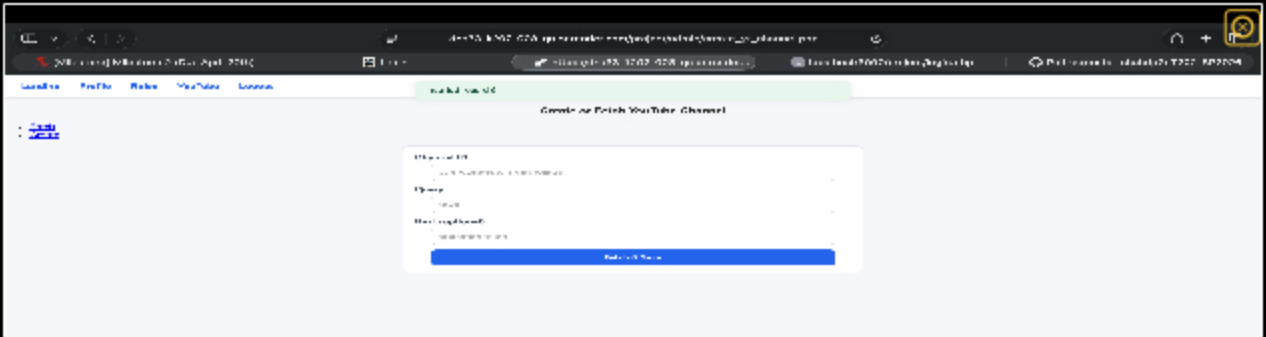
- Show the code that handles the data transformation/conversion (from the API)
- Show a Render.com qa example of API getting fetched and loaded into the DB

```
IT202_M2_YT_Channels  create_yt_channel.php  youtube_api.php
llb > youtube_api.php > C:\yt transform channel
30 4. NOTE: channel id is NOT always returned by some endpoints. If your endpoint
31 4 doesn't return it, we store the one you searched with.
32 4 dnw33
33 4
34 4 function yt_transform_channel($apijson, $channelid)
35 4 {
36 4     return [
37 4         "channel_id" => $channelid,
38 4         "title"      => $apijson["title"] ?? "",
39 4         "vanity_url"  => $apijson["vanityChannelUrl"] ?? null,
40 4         "verified"    => !empty($apijson["verified"]) ? 1 : 0,
41 4         "subscribers_text" => $apijson["subscriberCountText"] ?? null,
42 4         "avatar_url"  => yt_pick_best_thumbnail($apijson["avatar"] ?? []),
43 4         "is_api"      => 1,
44 4     ];
45 4 }
46 4
```

snippet -1

```
IT202_M2_YT_Channels  create_yt_channel.php  youtube_api.php
30 4 function yt_transform_video($apijson, $video_id, $channel_id)
31 4 {
32 4     return [
33 4         "video_id" => $video_id,
34 4         "channel_id" => $channel_id,
35 4         "title"      => $apijson["title"] ?? "",
36 4         "length_text" => $apijson["lengthText"] ?? "",
37 4         "published_text" => $apijson["publishedText"] ?? "",
38 4         "views_text"  => $apijson["viewCountText"] ?? "",
39 4         "thumbnail_url" => yt_pick_best_thumbnail($apijson["thumbnails"] ?? []),
40 4         "is_api"      => 1,
41 4     ];
42 4 }
43 4
```


snippet -2



webpage of insert of record 5

The screenshot shows a PostgreSQL database interface with a table named 'channels'. The table has the following columns: id, channel_id, title, vanity_url, verified, subscribers_count, avatar_url, created, and modified. The table contains three rows of data.

id	channel_id	title	vanity_url	verified	subscribers_count	avatar_url	created	modified
1	W4M0U_4_L001	My Channel	https://www.example.com/1	0	0	https://www.example.com/1	2023-04-22 10:34:15	2023-04-22 10:34:15
2	APL_C_L001	A1 Sample Channel	https://www.example.com/2	1	1000	https://www.example.com/2	2023-04-22 10:37:58	2023-04-22 10:37:58
3	UC_4S0G16G3F6173F03M0	Google's Developer	https://www.google.com/3	0	5,821,000,000	https://www.google.com/3	2023-04-24 10:34:40	2023-04-24 10:34:40

DB table look at id 5

 Saved: 4/26/2026 11:00:17 AM

➡ Part 2:

Progress: 100%

Details:

- Include Render.com prod link to API fetch page
- Include pull request link for this feature

- Details:**

 - Include Render.com prod link to API fetch page
 - Include pull request link for this feature

URL #1		URL https://dns33-it202-008-prod.onrender.com/project/admin/create_yt_channel.php	
URL #2		URL https://github.com/shahdp2/IT202-SP2026/pull/96	

URL #1		URL https://dns33-it202-008-prod.onrender.com/project/admin/create_yt_channel.php	
URL #2		URL https://github.com/shahdp2/IT202-SP2026/pull/96	

URL #2 <https://github.com/shahdp2/IT202-SP2026/pull/96>

URL #2 <https://github.com/shahdp2/IT202-SP2026/pull/96>

 Saved: 4/26/2026 11:00:17 AM

Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/extraction logic to get the API data to the shape your application intends to use

Your Response:

I am using a subset of the API response, not everything. I only store the fields needed for my app's channel and video tables.

Channel data (IT202_M2_YT_Channels): From the API JSON, I extract and map: title, vanityChannelUrl, verified, subscriberCountText, and the channel avatar thumbnail. The API does not reliably return channel_id in the payload, so I store the channel_id that the user searched with. I also convert verified from true/false into 1/0 for the database.

Video data (IT202_M2_YT_Videos): The API returns a contents array with different types of items, so I filter it and only keep items that contain a "video" object. I only insert videos that have a valid videoId (used as the unique ID). For each video I store: videoId, title, channelName, lengthText, publishedTimeText, viewCountText, and one thumbnail URL.



Saved: 4/26/2026 11:00:17 AM

⇒ Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
 - What happens if you get the same result from the API for an entity that exists but has been unmodified?
 - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

Duplicates are handled using UNIQUE constraints in the database and upsert logic in the insert queries.

The insert hits the UNIQUE key (channel_id for channels, video_id for videos) and runs the ON DUPLICATE KEY UPDATE path. Since the values are the same, the record effectively stays the same (at most modified may update, depending on how MySQL treats the update).

With the current design, the upsert will overwrite the stored fields with the newest API values because the duplicate update sets columns to VALUES(column). This keeps the DB aligned with the API, but it means manual edits can be replaced the next time the API fetch runs.

Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
 - (i.e., periodic via cron job, poor man's cron job, on app start/awake, etc)
 - (i.e., User or Admin triggered via a specific page/action)
 - (i.e., Lazy loaded during searches for stale/missing content)

Your Response:

API fetching is Admin-triggered and on-demand, not automatic. The fetch happens when an Admin uses the Create/Fetch page and submits the "Fetch (API)" form (channel id + query). That action calls the wrapper yt_channel_search() which fetches from the API, transforms the data, and inserts it into the DB.

This is not a cron job or automatic background fetch (to avoid wasting quota). It's only fetched when needed. This is also lazy because the app only requests API data for the specific channel/query the Admin asks for, instead of downloading large amounts of content all at once.

Section #2: (150 pts.) Data Creation

Progress: 100%

Task #1 (60 pts.) - Validations

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the html validations (code)
- Show the javascript validations (code)
- Show the php validations (code)

```

create_yt_channel.php M  create_yt_video.php M
public_html > project > admin > create_yt_channel.php > script
165 <!--div-->
166
167 <script>
168 // UCTD: dash
169 function switchTab(tab) {
170     let targets = document.getElementsByClassName("tab-target");
171     for (let ele of targets) {
172         ele.style.display = (ele.id === tab) ? "block" : "none";
173     }
174 }
175
176 </script>
177
178 </div>
179
180 </div>
181
182 </div>
183
184 </div>
185
186 </div>
187
188 </div>
189
190 </div>
191
192 </div>
193
194 </div>
195
196 </div>
197
198 </div>
199
200 </div>
201
202 </div>
203
204 </div>
205
206 </div>
207
208 </div>
209
210 </div>
211
212 </div>
213
214 </div>
215
216 </div>
217
218 </div>
219
220 </div>
221
222 </div>
223
224 </div>
225
226 </div>
227
228 </div>
229
230 </div>
231
232 </div>
233
234 </div>
235
236 </div>
237
238 </div>
239
240 </div>
241
242 </div>
243
244 </div>
245
246 </div>
247
248 </div>
249
250 </div>
251
252 </div>
253
254 </div>
255
256 </div>
257
258 </div>
259
260 </div>
261
262 </div>
263
264 </div>
265
266 </div>
267
268 </div>
269
270 </div>
271
272 </div>
273
274 </div>
275
276 </div>
277
278 </div>
279
280 </div>
281
282 </div>
283
284 </div>
285
286 </div>
287
288 </div>
289
290 </div>
291
292 </div>
293
294 </div>
295
296 </div>
297
298 </div>
299
300 </div>
301
302 </div>
303
304 </div>
305
306 </div>
307
308 </div>
309
310 </div>
311
312 </div>
313
314 </div>
315
316 </div>
317
318 </div>
319
320 </div>
321
322 </div>
323
324 </div>
325
326 </div>
327
328 </div>
329
330 </div>
331
332 </div>
333
334 </div>
335
336 </div>
337
338 </div>
339
340 </div>
341
342 </div>
343
344 </div>
345
346 </div>
347
348 </div>
349
350 </div>
351
352 </div>
353
354 </div>
355
356 </div>
357
358 </div>
359
360 </div>
361
362 </div>
363
364 </div>
365
366 </div>
367
368 </div>
369
370 </div>
371
372 </div>
373
374 </div>
375
376 </div>
377
378 </div>
379
380 </div>
381
382 </div>
383
384 </div>
385
386 </div>
387
388 </div>
389
390 </div>
391
392 </div>
393
394 </div>
395
396 </div>
397
398 </div>
399
400 </div>
401
402 </div>
403
404 </div>
405
406 </div>
407
408 </div>
409
410 </div>
411
412 </div>
413
414 </div>
415
416 </div>
417
418 </div>
419
420 </div>
421
422 </div>
423
424 </div>
425
426 </div>
427
428 </div>
429
430 </div>
431
432 </div>
433
434 </div>
435
436 </div>
437
438 </div>
439
440 </div>
441
442 </div>
443
444 </div>
445
446 </div>
447
448 </div>
449
450 </div>
451
452 </div>
453
454 </div>
455
456 </div>
457
458 </div>
459
460 </div>
461
462 </div>
463
464 </div>
465
466 </div>
467
468 </div>
469
470 </div>
471
472 </div>
473
474 </div>
475
476 </div>
477
478 </div>
479
480 </div>
481
482 </div>
483
484 </div>
485
486 </div>
487
488 </div>
489
490 </div>
491
492 </div>
493
494 </div>
495
496 </div>
497
498 </div>
499
500 </div>
501
502 </div>
503
504 </div>
505
506 </div>
507
508 </div>
509
510 </div>
511
512 </div>
513
514 </div>
515
516 </div>
517
518 </div>
519
520 </div>
521
522 </div>
523
524 </div>
525
526 </div>
527
528 </div>
529
530 </div>
531
532 </div>
533
534 </div>
535
536 </div>
537
538 </div>
539
540 </div>
541
542 </div>
543
544 </div>
545
546 </div>
547
548 </div>
549
550 </div>
551
552 </div>
553
554 </div>
555
556 </div>
557
558 </div>
559
560 </div>
561
562 </div>
563
564 </div>
565
566 </div>
567
568 </div>
569
570 </div>
571
572 </div>
573
574 </div>
575
576 </div>
577
578 </div>
579
580 </div>
581
582 </div>
583
584 </div>
585
586 </div>
587
588 </div>
589
590 </div>
591
592 </div>
593
594 </div>
595
596 </div>
597
598 </div>
599
600 </div>
601
602 </div>
603
604 </div>
605
606 </div>
607
608 </div>
609
610 </div>
611
612 </div>
613
614 </div>
615
616 </div>
617
618 </div>
619
620 </div>
621
622 </div>
623
624 </div>
625
626 </div>
627
628 </div>
629
630 </div>
631
632 </div>
633
634 </div>
635
636 </div>
637
638 </div>
639
640 </div>
641
642 </div>
643
644 </div>
645
646 </div>
647
648 </div>
649
650 </div>
651
652 </div>
653
654 </div>
655
656 </div>
657
658 </div>
659
660 </div>
661
662 </div>
663
664 </div>
665
666 </div>
667
668 </div>
669
670 </div>
671
672 </div>
673
674 </div>
675
676 </div>
677
678 </div>
679
680 </div>
681
682 </div>
683
684 </div>
685
686 </div>
687
688 </div>
689
690 </div>
691
692 </div>
693
694 </div>
695
696 </div>
697
698 </div>
699
700 </div>
701
702 </div>
703
704 </div>
705
706 </div>
707
708 </div>
709
710 </div>
711
712 </div>
713
714 </div>
715
716 </div>
717
718 </div>
719
720 </div>
721
722 </div>
723
724 </div>
725
726 </div>
727
728 </div>
729
730 </div>
731
732 </div>
733
734 </div>
735
736 </div>
737
738 </div>
739
740 </div>
741
742 </div>
743
744 </div>
745
746 </div>
747
748 </div>
749
750 </div>
751
752 </div>
753
754 </div>
755
756 </div>
757
758 </div>
759
760 </div>
761
762 </div>
763
764 </div>
765
766 </div>
767
768 </div>
769
770 </div>
771
772 </div>
773
774 </div>
775
776 </div>
777
778 </div>
779
780 </div>
781
782 </div>
783
784 </div>
785
786 </div>
787
788 </div>
789
790 </div>
791
792 </div>
793
794 </div>
795
796 </div>
797
798 </div>
799
800 </div>
801
802 </div>
803
804 </div>
805
806 </div>
807
808 </div>
809
810 </div>
811
812 </div>
813
814 </div>
815
816 </div>
817
818 </div>
819
820 </div>
821
822 </div>
823
824 </div>
825
826 </div>
827
828 </div>
829
830 </div>
831
832 </div>
833
834 </div>
835
836 </div>
837
838 </div>
839
840 </div>
841
842 </div>
843
844 </div>
845
846 </div>
847
848 </div>
849
850 </div>
851
852 </div>
853
854 </div>
855
856 </div>
857
858 </div>
859
860 </div>
861
862 </div>
863
864 </div>
865
866 </div>
867
868 </div>
869
870 </div>
871
872 </div>
873
874 </div>
875
876 </div>
877
878 </div>
879
880 </div>
881
882 </div>
883
884 </div>
885
886 </div>
887
888 </div>
889
890 </div>
891
892 </div>
893
894 </div>
895
896 </div>
897
898 </div>
899
900 </div>
901
902 </div>
903
904 </div>
905
906 </div>
907
908 </div>
909
910 </div>
911
912 </div>
913
914 </div>
915
916 </div>
917
918 </div>
919
920 </div>
921
922 </div>
923
924 </div>
925
926 </div>
927
928 </div>
929
930 </div>
931
932 </div>
933
934 </div>
935
936 </div>
937
938 </div>
939
940 </div>
941
942 </div>
943
944 </div>
945
946 </div>
947
948 </div>
949
950 </div>
951
952 </div>
953
954 </div>
955
956 </div>
957
958 </div>
959
960 </div>
961
962 </div>
963
964 </div>
965
966 </div>
967
968 </div>
969
970 </div>
971
972 </div>
973
974 </div>
975
976 </div>
977
978 </div>
979
980 </div>
981
982 </div>
983
984 </div>
985
986 </div>
987
988 </div>
989
990 </div>
991
992 </div>
993
994 </div>
995
996 </div>
997
998 </div>
999
1000 </div>

```

```
173 }
174 }
175 </script>
176
177 <?php require_once($_DIR_ . "/../..../partials/flash.php"); ?>
```

JS validation - 1

```
173 // validate video id
174 function validateVideoId() {
175     var videoId = document.getElementById("video_id").value.trim();
176     if (videoId.length < 11) {
177         alert("Video ID must be at least 11 characters");
178         return false;
179     }
180     return true;
181 }
182
183 <?php require_once($_DIR_ . "/../..../partials/flash.php"); ?>
```

JS validation - 2

```
173 // validate video id
174 function validateVideoId() {
175     var videoId = document.getElementById("video_id").value.trim();
176     if (videoId.length < 11) {
177         alert("Video ID must be at least 11 characters");
178         return false;
179     }
180     return true;
181 }
182
183 <?php require_once($_DIR_ . "/../..../partials/flash.php"); ?>
```

PHP file 1 validation

```
173 // validate video id
174 function validateVideoId() {
175     var videoId = document.getElementById("video_id").value.trim();
176     if (videoId.length < 11) {
177         alert("Video ID must be at least 11 characters");
178         return false;
179     }
180     return true;
181 }
182
183 <?php require_once($_DIR_ . "/../..../partials/flash.php"); ?>
```

PHP file 2 validation

```
173 // validate video id
174 function validateVideoId() {
175     var videoId = document.getElementById("video_id").value.trim();
176     if (videoId.length < 11) {
177         alert("Video ID must be at least 11 characters");
178         return false;
179     }
180     return true;
181 }
182
183 <?php require_once($_DIR_ . "/../..../partials/flash.php"); ?>
```

HTML file 1 validation(1)

```
173 // validate video id
174 function validateVideoId() {
175     var videoId = document.getElementById("video_id").value.trim();
176     if (videoId.length < 11) {
177         alert("Video ID must be at least 11 characters");
178         return false;
179     }
180     return true;
181 }
182
183 <?php require_once($_DIR_ . "/../..../partials/flash.php"); ?>
```

HTML file 1 validation(2)

HTML file 2 validation(1)

HTML file 2 validation(2)

≡ Part 2:

Details:

- Your Response:**

HTML validations: I used built-in HTML form validation to block obviously invalid submissions before they reach the server. The manual create forms mark required fields with required (Channel: channel_id, title; Video: video_id, channel_id, title, channel_name). I also used proper

input types like type="url" for URL fields (vanity/avatar/thumbnail) and maxlength limits to match the database column sizes.

JavaScript validation logic: Before the form submits, JavaScript performs extra client-side checks that are specific to YouTube data (for example, validating the channel_id/video_id format/length). If the values don't meet the expected pattern, the script prevents submission and shows a user-friendly error message so the user can fix it immediately.

PHP validation logic: On the server side, I validate again for security and correctness. I whitelist allowed POST keys (only columns that exist in the table) and ignore anything else. I check required fields again (channel_id/title for channels; video_id/channel_id/title for videos). I also normalize checkbox values like verified into 1/0 and force is_api=0 for manual entries. If validation fails, the record is not inserted and an appropriate flash message is shown.

Access / roles: This create/fetch page is restricted to the Admin role only (non-admins are blocked from the page).



Saved: 4/27/2026 3:39:58 PM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a successful creation
- Show how a duplicate submission is handled (i.e., what message or behavior occurs)
- Demonstrate various validation errors
- Ensure Render.com qa url is visible



Success Video Creation look at id 5

	id	channel_id	video_id	title	description	thumbnail_url	model_name	verified	created_at	updated_at	deleted_at
1	1	channel_1	video_1	Video Title 1	Video Description 1	https://example.com/thumbnail1.jpg	Model Name 1	1	2023-04-21 10:30:00	2023-04-21 10:30:00	
2	2	channel_2	video_2	Video Title 2	Video Description 2	https://example.com/thumbnail2.jpg	Model Name 2	0	2023-04-21 11:30:00	2023-04-21 11:30:00	
3	3	channel_3	video_3	Video Title 3	Video Description 3	https://example.com/thumbnail3.jpg	Model Name 3	1	2023-04-21 12:30:00	2023-04-21 12:30:00	
4	4	channel_4	video_4	Video Title 4	Video Description 4	https://example.com/thumbnail4.jpg	Model Name 4	0	2023-04-21 13:30:00	2023-04-21 13:30:00	
5	5	channel_5	video_5	Video Title 5	Video Description 5	https://example.com/thumbnail5.jpg	Model Name 5	1	2023-04-21 14:30:00	2023-04-21 14:30:00	

Look at id 5

Look at id 5

CREATE OR PATCH YOUTUBE VIDEOS

Video ID:

Title:

Description:

Tags:

Buttons:

duplicate

CREATE OR PATCH YOUTUBE VIDEOS

Video ID:

Title:

Description:

Tags:

Buttons:

HTML error

CREATE OR PATCH YOUTUBE VIDEOS

Video ID:

Title:

Description:

Tags:

Buttons:

JS Error

CREATE OR PATCH YOUTUBE VIDEOS

Video ID:

Title:

Description:

Tags:

Buttons:

PHP Error



Saved: 4/24/2026 5:06:01 PM

⇒ Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://dns33-it202-008-prod.onrender.com/project/admin/create_yt_videos.php

URL

https://dns33-it202-008-prod.



URL #2

<https://github.com/shahdp2/IT202-SP2026/pull/97>

URL

<https://github.com/shahdp2/>



Saved: 4/24/2026 5:03:19 PM

Section #3: (150 pts.) Data List Page (Many Items)

Progress: 100%

≡ Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the filter/sort logic
- Show the code that enforces the record limit (valid range 1-100, default 10 if out of range)
- Show the code that generates the list/grid output
- Styling must be applied

[illegible]

snippet -1

CC BY-NC-ND 4.0 International license arXiv:1604.04702v1 [hep-th] 1 Apr 2016

matches the search term being entered. The search value is bound to a parameter (something like `%search%`) so it's safe and not vulnerable to SQL injection. For sorting, the page reads sort and dir, but the sort column is restricted to a whitelist of allowed columns; if the user passes anything else, it falls back to a default (like modified). The sort direction is also normalized so only asc is allowed, otherwise it defaults to desc, and the query ends with `ORDER BY $sort $dir`.

The record limit is validated server-side. The page reads limit from `$_GET`, converts it to an integer, and enforces the required range of 1–100. If the value is out of range or invalid, it defaults to 10. The limit is then applied in SQL using `LIMIT :lim`, and it's bound using `PDO::PARAM_INT`, which ensures it's treated strictly as a number.

After executing the query, the output is generated dynamically from the result set. The code fetches rows using `fetchAll(PDO::FETCH_ASSOC)` and then loops through the array to build an HTML table. The first row is used to print the column headers, and each subsequent row prints the record values in table cells. An "Actions" column is appended for each row, using the record's id in the query string so the user can navigate to View/Edit/Delete for that specific record.

Styling is kept consistent and readable by wrapping everything in a container (container-fluid), spacing the filter controls using mb-3, and applying a table class (class="table") for a clean table layout. This keeps the filter controls easy to use and makes the list readable even when there are many records.



Saved: 4/26/2026 11:00:43 AM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a few variations of filter/sort applied (including no records - should display a "no results" message)
- Show both API-fetched and manually created entities present in the list
- Show the limit field in use with at least two different values
- Each list item should be a summary
- Each list item should have the following links
 - A link to a single view of the specific entity (i.e., a details page)
 - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
 - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure Render.com qa url is visible



This single screenshot checks: API+manual + actions links + prod URL visible.



Proof filtering works



"No results to show" message + prod URL.



Proof sort/dir work



Limit works





Another proof of limit working



Saved: 4/25/2026 10:12:13 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://dns33-it202-008-prod.onrender.com/project/admin/list_yt_channels.php



URL

[https://dns33-it202-008-prod.](https://dns33-it202-008-prod.onrender.com/project/admin/list_yt_channels.php)



URL #2

<https://github.com/shahdp2/IT202-SP2026/pull/105>



URL

[https://github.com/shahdp2/](https://github.com/shahdp2/IT202-SP2026/pull/105)



Saved: 4/25/2026 9:48:43 PM

Section #4: (150 pts.) View Details Page (Single Item)

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect back to the previous page with an appropriate message)
- Show the code that generates the entity output
- Styling must be applied



proves loading + id usage + invalid handling



handling everything

 Saved: 4/26/2026 11:00:52 AM

⇒, Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the entity output is generated
- Briefly note the styling choices

- Details:**

 - Briefly explain how the loading logic works
 - Briefly explain how the entity output is generated
 - Briefly note the styling choices

Your Response:

ALL YOUTUBE CRUD OPERATIONS ARE ADMIN ONLY.

The view page reads the channel id from \$_GET, validates it, and either redirects back to the list with a flash message (if missing/invalid/not found) or runs a prepared SELECT ... WHERE id = :id to load the record. It then outputs the channel fields using se(...) for safe display, and uses the shared styles.css/container styling so the details are clean and consistent with the rest of the site.

ALL YOUTUBE CRUD OPERATIONS ARE ADMIN ONLY.

The view page reads the channel id from \$_GET, validates it, and either redirects back to the list with a flash message (if missing/invalid/not found) or runs a prepared SELECT ... WHERE id = :id to load the record. It then outputs the channel fields using se(...) for safe display, and uses the shared styles.css/container styling so the details are clean and consistent with the rest of the site.

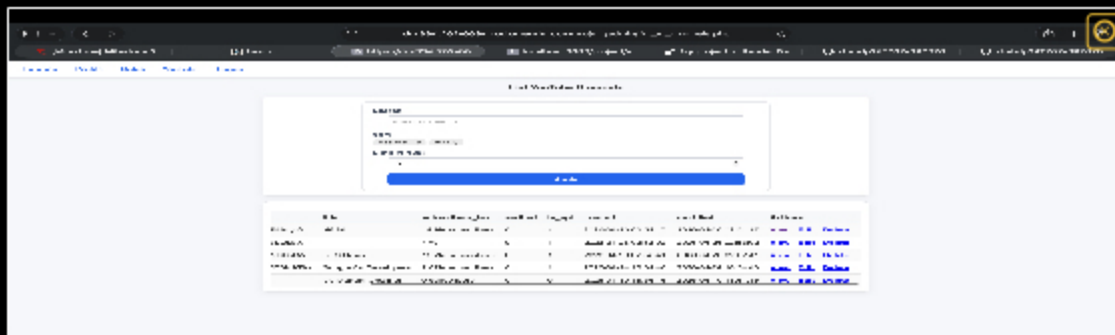
 Saved: 4/26/2026 11:00:52 AM

Task #2 (60 pts.) - Examples

Task #2 (60 pts.) - Examples

Details:

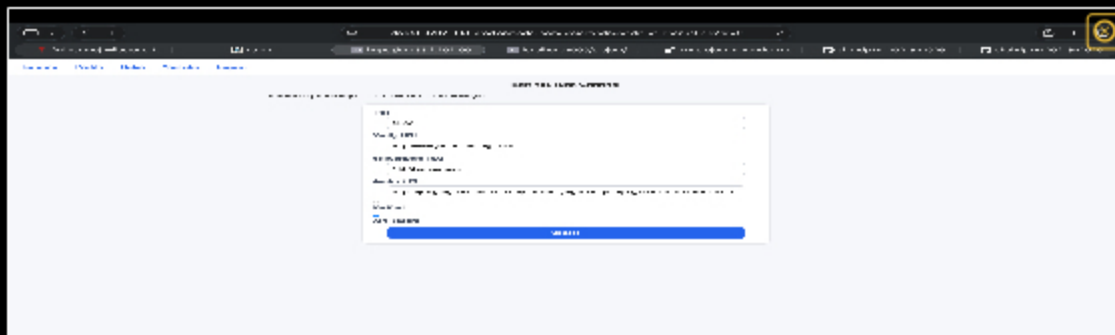
- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
 - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
 - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)



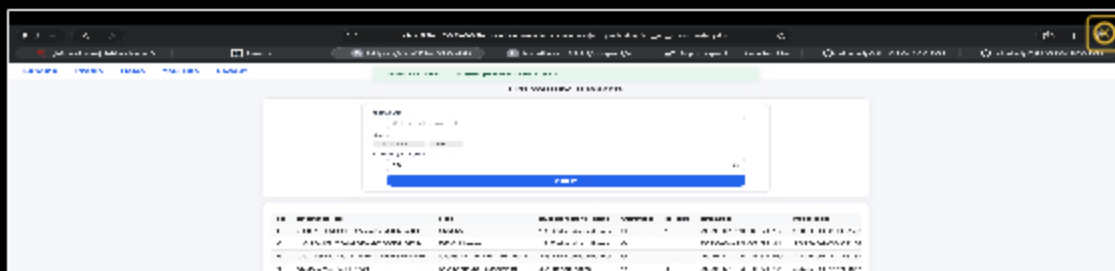
View/Edit/Delete Entities



View page



Edit Page



Delete



No result show



Saved: 4/25/2026 11:16:06 PM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://dns33-it202-008-prod.onrender.com/project/admin/list_yt_channels.php



URL

https://dns33-it202-008-prod.onrender.com/project/admin/list_yt_channels.php



URL #2

<https://github.com/shahdp2/IT202-SP2026/pull/108>



URL

<https://github.com/shahdp2/IT202-SP2026/pull/108>



Saved: 4/25/2026 11:16:49 PM

Section #5: (150 pts.) Edit Page

Progress: 100%

Task #1 (60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)

- [illegible]

file 1 - PHP

```
file 1 -html
```

file -1 JS

file -2 PHP

[illegible]

provided, the code runs a SELECT ... WHERE id = :id to fetch the current row and populate the form. If no row is found, it flashes "not found" and redirects so users don't see a blank/broken edit page.

When the form is submitted (POST), the server first whitelists the allowed fields (so only approved columns can be updated). It then normalizes checkbox values (ex: verified / is_api into 1/0) and calls the update() helper to run a parameterized UPDATE using the current record id. After updating, it reloads the record and shows a success message if a change happened, or a "No changes made" warning if the submitted values match what's already stored.

Styling is done with simple Bootstrap-style spacing (container-fluid, mb-3, and button classes) so the form is readable and consistent with the rest of the site. Only "safe" fields are editable (ex: titles, URLs, subscriber/view text, etc.), while identity/system fields like id, created, modified, and unique identifiers (channel_id, video_id) are read-only to prevent breaking references and to keep the database consistent when re-fetching from the API.

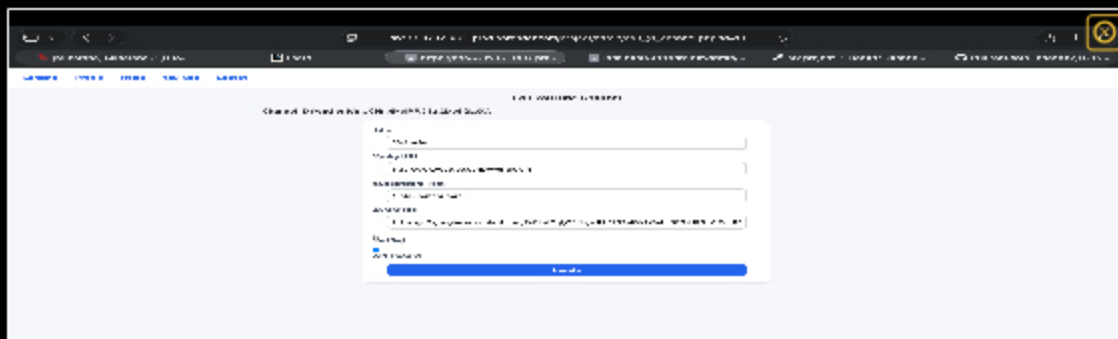
 Saved: 4/26/2026 11:01:01 AM

Task #2 (60 pts.) - Examples

Progress: 100%

Details:

- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown
- Show some examples of validation errors (html, javascript, php)



Channel Details

Title

Description

URL

Subscriber Count

View Count

Verified

API

Update

Before Edit



Channel Details

Title

Description

URL

Subscriber Count

View Count

Verified

API

Update

After Update

Basic Task Submitted

Title: Task

Category: Task

Description: This is a description of the task

Author: John Doe

Submit

No Changes Made

Basic Task Submitted

Title:

Category: Task

Description: This is a description of the task

Author: John Doe

Submit

HTML validation blocks submission when required field is empty.

Basic Task Submitted

The title is required.

Title:

Category: Task

Description: This is a description of the task

Author: John Doe

Submit

JS shows "Avatar URL must start with https://"

Basic Task Submitted

The avatar URL must start with https://.

Title: Task

Category: Task

Description: This is a description of the task

Author: John Doe

Submit

PHP validation title can't be blank



Saved: 4/26/2026 12:24:53 AM

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://dns33-it202-008-prod.onrender.com/project/admin/edit_yt_channel.php?id=10



URL

[https://dns33-it202-008-prod.](https://dns33-it202-008-prod.onrender.com/project/admin/edit_yt_channel.php?id=10)
**URL #2**

<https://github.com/shahdp2/IT202-SP2026/pull/117>



URL

[https://github.com/shahdp2/](https://github.com/shahdp2/IT202-SP2026/pull/117)


Saved: 4/26/2026 12:25:45 AM

Section #6: (150 pts.) Delete Logic

Progress: 100%

≡ Task #1 (60 pts.) - Code

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect to the list page with an appropriate message)
- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/sort criteria

```

<?php
// ... (previous code) ...

function deleteRecord($id, $redirectUrl) {
    // Check if ID is valid
    if ($id < 1) {
        // Invalid ID, redirect to list page with message
        header("Location: " . $redirectUrl . "?message=Invalid ID");
        exit();
    }

    // Delete record from database
    $sql = "DELETE FROM records WHERE id = $id";
    $result = mysqli_query($conn, $sql);

    if ($result === TRUE) {
        // Record deleted successfully, redirect to list page
        header("Location: " . $redirectUrl);
        exit();
    } else {
        // Error deleting record, redirect to list page with message
        header("Location: " . $redirectUrl . "?message=Error deleting record");
        exit();
    }
}

// ... (main script code) ...
deleteRecord($_GET['id'], $_SERVER['HTTP_REFERER']);

```

code -1



code -2



⇒, Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/sort data

- Details:**

 - Briefly explain how the deletion logic works
 - Is it a soft or hard delete?
 - Note any permissions/rules applied.
 - Briefly explain how the redirect maintains the previous route and any filter/sort data

Your Response:

ALL YOUTUBE CRUD OPERATIONS ARE ADMIN ONLY.

The delete pages are hard deletes. When the user clicks a Delete link, the page reads the record id from `$_GET["id"]`, validates it (`id >= 1`), and checks that the user has the Admin role before doing anything. If the id is missing/invalid or the user is not an Admin, the code flashes a warning and redirects safely back to a valid page.

If the request is valid, the code runs a `DELETE FROM ... WHERE id = :id` prepared statement (so it's safe from SQL injection). After deletion, it flashes either a success message (deleted) or a warning message (not found / already deleted).

For redirects, the delete link includes a return URL (ex: `?return=...`) that contains the previous list page + its current query string (search/sort/dir/limit). After the delete runs, the code redirects back to that return URL, so the list reloads with the same filter/sort/limit settings the user had before.

ALL YOUTUBE CRUD OPERATIONS ARE ADMIN ONLY.

The delete pages are hard deletes. When the user clicks a Delete link, the page reads the record id from `$_GET["id"]`, validates it (`id >= 1`), and checks that the user has the Admin role before doing anything. If the id is missing/invalid or the user is not an Admin, the code flashes a warning and redirects safely back to a valid page.

If the request is valid, the code runs a `DELETE FROM ... WHERE id = :id` prepared statement (so it's safe from SQL injection). After deletion, it flashes either a success message (deleted) or a warning message (not found / already deleted).

For redirects, the delete link includes a return URL (ex: `?return=...`) that contains the previous list page + its current query string (search/sort/dir/limit). After the delete runs, the code redirects back to that return URL, so the list reloads with the same filter/sort/limit settings the user had before.

ALL YOUTUBE CRUD OPERATIONS ARE ADMIN ONLY.

The delete pages are hard deletes. When the user clicks a Delete link, the page reads the record id from `$_GET["id"]`, validates it (`id >= 1`), and checks that the user has the Admin role before doing anything. If the id is missing/invalid or the user is not an Admin, the code flashes a warning and redirects safely back to a valid page.

If the request is valid, the code runs a `DELETE FROM ... WHERE id = :id` prepared statement (so it's safe from SQL injection). After deletion, it flashes either a success message (deleted) or a warning message (not found / already deleted).

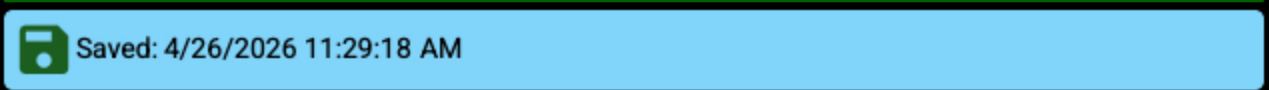
For redirects, the delete link includes a return URL (ex: `?return=...`) that contains the previous list page + its current query string (search/sort/dir/limit). After the delete runs, the code redirects back to that return URL, so the list reloads with the same filter/sort/limit settings the user had before.

ALL YOUTUBE CRUD OPERATIONS ARE ADMIN ONLY.

The delete pages are hard deletes. When the user clicks a Delete link, the page reads the record id from `$_GET["id"]`, validates it (`id >= 1`), and checks that the user has the Admin role before doing anything. If the id is missing/invalid or the user is not an Admin, the code flashes a warning and redirects safely back to a valid page.

If the request is valid, the code runs a `DELETE FROM ... WHERE id = :id` prepared statement (so it's safe from SQL injection). After deletion, it flashes either a success message (deleted) or a warning message (not found / already deleted).

For redirects, the delete link includes a return URL (ex: `?return=...`) that contains the previous list page + its current query string (search/sort/dir/limit). After the delete runs, the code redirects back to that return URL, so the list reloads with the same filter/sort/limit settings the user had before.



Task #2 (60 pts.) - Examples Progress: 100%

Task #2 (60 pts.) - Examples Progress: 100%

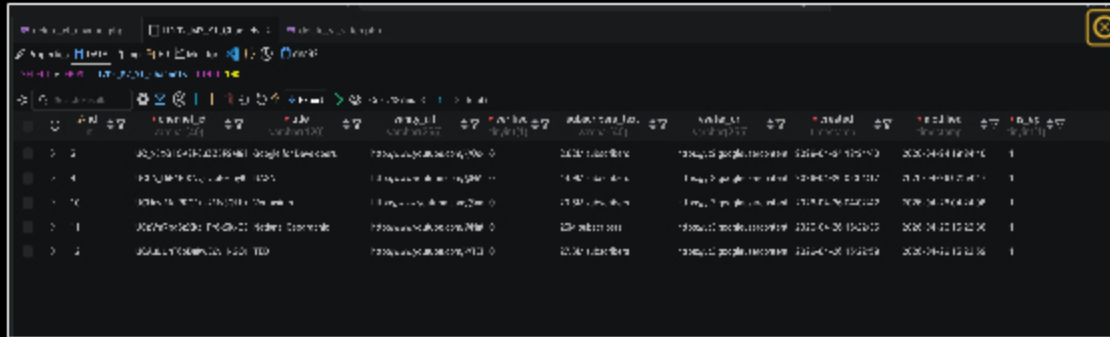
Details:

- Show a successful delete message

- Details:**

 - Show a successful delete message

- Show a before and after database view of the record being deleted (clearly caption)
- Any errors should have user-friendly messages shown



Screenshot of a database table showing records before deletion. The table has columns: ID, Name, Email, Password, Age, Gender, Address, City, State, Zip, and Active. There are 5 records displayed.

ID	Name	Email	Password	Age	Gender	Address	City	State	Zip	Active
1	John Doe	john.doe@example.com	password123	30	Male	123 Main St	New York	NY	10001	1
2	Jane Smith	jane.smith@example.com	password456	25	Female	456 Elm St	Los Angeles	CA	90001	1
3	Bob Johnson	bob.johnson@example.com	password789	35	Male	789 Oak St	Chicago	IL	60601	1
4	Alice Brown	alice.brown@example.com	password012	28	Female	012 Pine St	San Francisco	CA	94101	1
5	Charlie Davis	charlie.davis@example.com	password345	32	Male	345 Maple St	Seattle	WA	98101	1

DB before Deletion



Screenshot of a web application showing a table of users before deletion. The table has columns: ID, Name, Email, Password, Age, Gender, Address, City, State, Zip, and Active. There are 5 records displayed.

ID	Name	Email	Password	Age	Gender	Address	City	State	Zip	Active
1	John Doe	john.doe@example.com	password123	30	Male	123 Main St	New York	NY	10001	1
2	Jane Smith	jane.smith@example.com	password456	25	Female	456 Elm St	Los Angeles	CA	90001	1
3	Bob Johnson	bob.johnson@example.com	password789	35	Male	789 Oak St	Chicago	IL	60601	1
4	Alice Brown	alice.brown@example.com	password012	28	Female	012 Pine St	San Francisco	CA	94101	1
5	Charlie Davis	charlie.davis@example.com	password345	32	Male	345 Maple St	Seattle	WA	98101	1

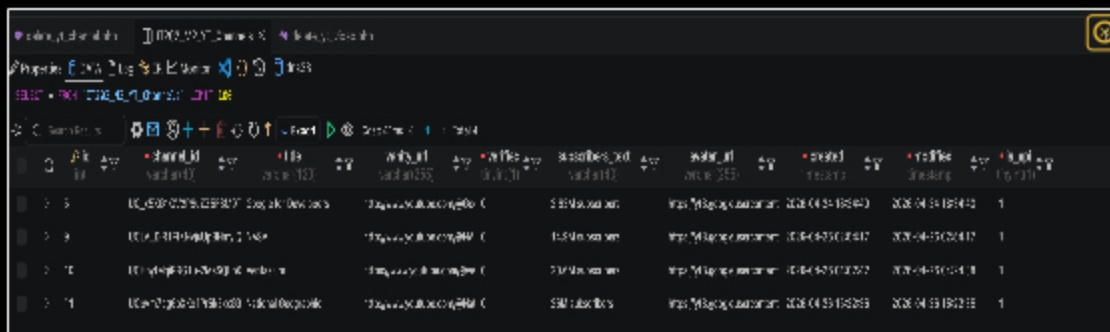
webpage before deletion



Screenshot of a web application showing a table of users after deletion. The table has columns: ID, Name, Email, Password, Age, Gender, Address, City, State, Zip, and Active. There are 4 records displayed, as the first record (John Doe) has been removed.

ID	Name	Email	Password	Age	Gender	Address	City	State	Zip	Active
2	Jane Smith	jane.smith@example.com	password456	25	Female	456 Elm St	Los Angeles	CA	90001	1
3	Bob Johnson	bob.johnson@example.com	password789	35	Male	789 Oak St	Chicago	IL	60601	1
4	Alice Brown	alice.brown@example.com	password012	28	Female	012 Pine St	San Francisco	CA	94101	1
5	Charlie Davis	charlie.davis@example.com	password345	32	Male	345 Maple St	Seattle	WA	98101	1

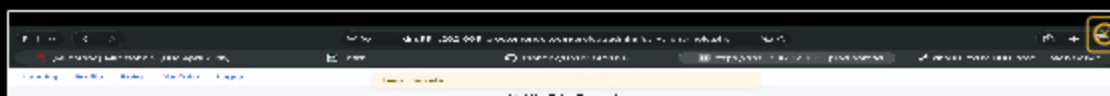
Webpage After deletion, see TED channel is gone



Screenshot of a database table showing records after deletion. The table has columns: ID, Name, Email, Password, Age, Gender, Address, City, State, Zip, and Active. There are 4 records displayed, as the first record (John Doe) has been removed.

ID	Name	Email	Password	Age	Gender	Address	City	State	Zip	Active
2	Jane Smith	jane.smith@example.com	password456	25	Female	456 Elm St	Los Angeles	CA	90001	1
3	Bob Johnson	bob.johnson@example.com	password789	35	Male	789 Oak St	Chicago	IL	60601	1
4	Alice Brown	alice.brown@example.com	password012	28	Female	012 Pine St	San Francisco	CA	94101	1
5	Charlie Davis	charlie.davis@example.com	password345	32	Male	345 Maple St	Seattle	WA	98101	1

DB After deletion, see TED channel is gone



Screenshot of a web application showing a table of users after deletion. The table has columns: ID, Name, Email, Password, Age, Gender, Address, City, State, Zip, and Active. There are 4 records displayed, as the first record (John Doe) has been removed.

ID	Name	Email	Password	Age	Gender	Address	City	State	Zip	Active
2	Jane Smith	jane.smith@example.com	password456	25	Female	456 Elm St	Los Angeles	CA	90001	1
3	Bob Johnson	bob.johnson@example.com	password789	35	Male	789 Oak St	Chicago	IL	60601	1
4	Alice Brown	alice.brown@example.com	password012	28	Female	012 Pine St	San Francisco	CA	94101	1
5	Charlie Davis	charlie.davis@example.com	password345	32	Male	345 Maple St	Seattle	WA	98101	1

Table 1: User Information					
User ID: 123456789					
Name: John Doe					
Email: john.doe@example.com					
Phone: (123) 456-7890					
Address: 123 Main St, New York, NY 10001					
Created At: 2023-01-01 10:00:00					
Updated At: 2023-01-01 10:00:00					
Status: Active					
Last Login: 2023-01-01 10:00:00					
Role: Admin					
Permissions: Read, Write, Delete					

user friendly error message

 Saved: 4/26/2026 11:38:08 AM

Task #3 (30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL #1

https://dns33-it202-008-prod.onrender.com/project/admin/list_yt_channels.php



URL

https://dns33-it202-008-prod.onrender.com/project/admin/list_yt_channels.php



URL #2

<https://github.com/shahdp2/IT202-SP2026/pull/120>



URL

<https://github.com/shahdp2/IT202-SP2026/pull/120>



 Saved: 4/26/2026 11:32:02 AM

Section #7: (50 pts.) Misc

Progress: 100%

Task #1 (16.67 pts.) - Github Details

Progress: 100%

Part 1:

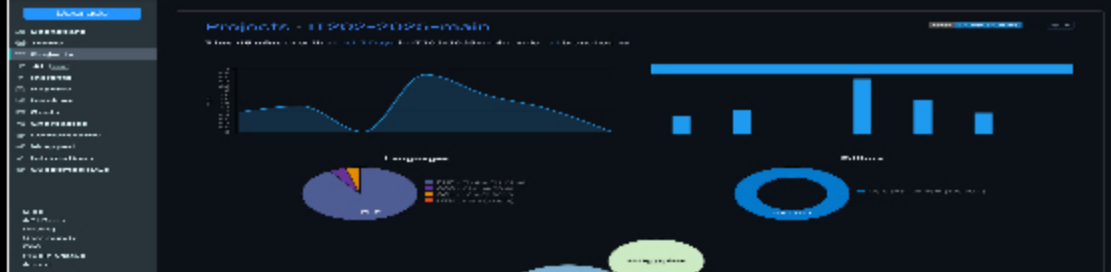
Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)







waketime -1



waketime -2



waketime -3

 Saved: 4/27/2026 3:38:52 PM

Task #3 (16.67 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

For Milestone 2, I learned how to take real API data and integrate it into a full CRUD workflow. I practiced designing database tables with proper types/constraints, transforming raw JSON into "DB-ready" rows, and handling

duplicates safely using UNIQUE keys and upsert logic. I also learned why server-side validation and permissions matter, since the UI can be bypassed but the backend still needs to protect the database. Finally, deploying on Render helped me understand how QA vs PROD environments can behave differently and why testing in deployed environments is important.



Saved: 4/26/2026 3:00:01 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was building the basic CRUD pages using the class templates as a reference. Once the tables existed, it was straightforward to create the forms and connect them to INSERT/UPDATE queries with a whitelist of allowed fields. The list pages were also manageable because the filter/sort/limit pattern was similar to what we already did in earlier lessons. Styling was also simple once I reused consistent classes and layouts across pages.



Saved: 4/26/2026 2:59:54 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was getting the API integration and data transformation correct and consistent across both channels and videos. The API response format had nested structures, so I had to carefully extract only the fields I needed and handle missing keys safely. Another challenge was handling duplicates and updates in a way that made sense, since repeated API fetches can overwrite manual edits if you always upsert. Debugging deployed issues on Render (like redirects/headers and differences between environments) also took the most time because it required careful testing and making sure redirects and flash messages worked correctly.

connect.



Saved: 4/26/2026 2:59:47 PM